Accountability in Multiagent Organizations from Concepts to Software Engineering

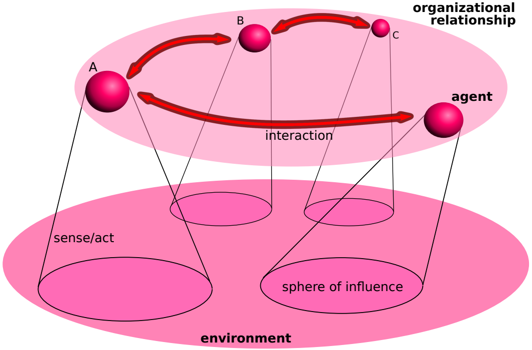
Matteo Baldoni[1], Cristina Baroglio[1], Elisa Marengo[1], Roberto Micalizio[1], Stefano Tedeschi[2]

June 1, 2026 Web Agent

[1]Università degli Studi di Torino, Computer Science Department, Italy
[2]Università della Valle d'Aosta - Université de la Vallée d'Aoste, Italy

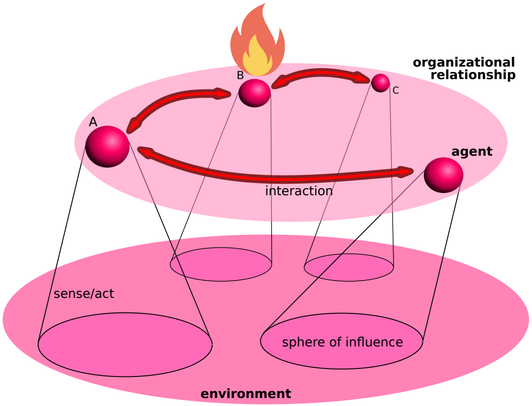
Multiagent Organizations

- *"An organization provides a structure of constraints that allow a system consisting of many parts to act as a whole, with the aim of achieving goals that otherwise would not be achievable (or not as easily)"* **ElderVass**
- **Norms** (rules, protocols, etc.) to define what is expected of each agent
- **Sanctions** as deterrents **to prevent** norm violation



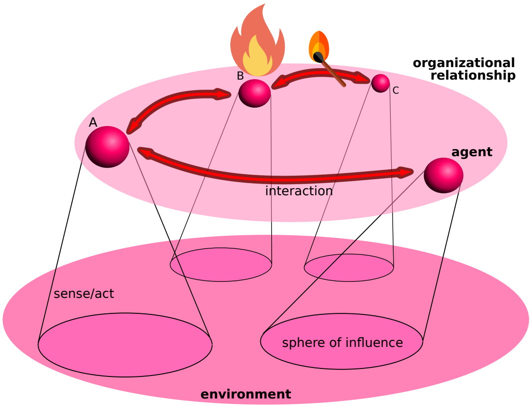
Perturbations

- Unfortunately, when the system faces an abnormal situation (**perturbation**) and some agent *fails* to achieve a goal, **sanctions are of little utility**, if any



Perturbations

- Unfortunately, when the system faces an abnormal situation (**perturbation**) and some agent *fails* to achieve a goal, **sanctions are of little utility**, if any
- The agent may have tried its best to do what expected, but something which is not under its control might hinder the achievement

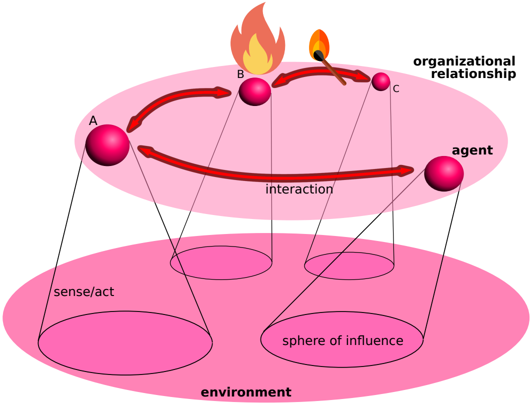


Perturbations

- Unfortunately, when the system faces an abnormal situation (**perturbation**) and some agent *fails* to achieve a goal, **sanctions are of little utility**, if any
- The agent may have tried its best to do what expected, but something which is not under its control might hinder the achievement

Broader problem

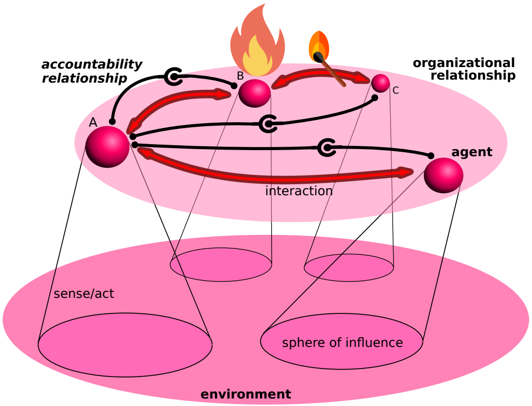
No structured way for collecting and propagating information about encountered situations



Robustness

When a system meets a perturbation it needs **to reconfigure**. To this aim:

1. the agents need the means for: understanding who is entitled to ask what to whom
2. the information of interest must be asked to an informed source and must be delivered in the right format
3. the information will be delivered to whom is equipped with the right abilities and will be entitled to perform certain tasks, needed to cope with the situation

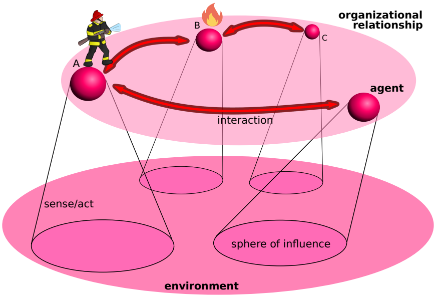


Robustness

Robustness

The degree to which a system or component can function correctly in presence of invalid inputs or stressful environmental conditions[1] – generally called **perturbations**

Robustness should be realized through appropriate distributions of responsibilities among the agents



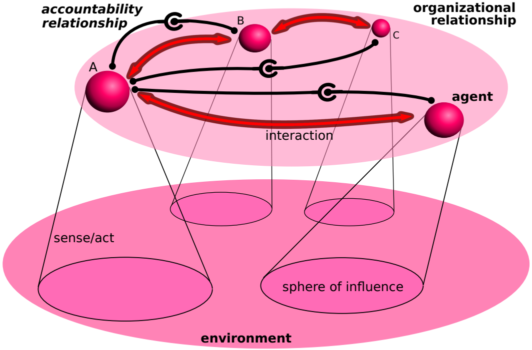
[1] **"ISO/IEC/IEEE International Standard - Systems and software engineering – Vocabulary".** In: *ISO/IEC/IEEE 24765:2010(E)* (2010), pp. 1–418.

Robustness and accountability

Claim

accountability provides the necessary tools.

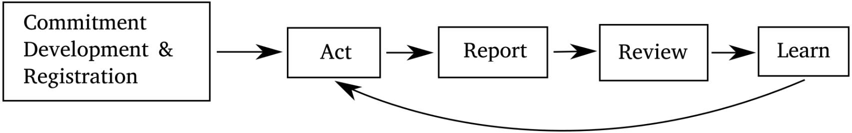
- We claim that **account** and **accountability** are the crucial tools for making organizations more **robust**
- In the **human world/organizations** accountability provides the means to address recurring and systemic issues, and to incorporate lessons learned into future activities



Accountability in the Human World

Accountability frameworks describe organization-wide processes for monitoring, analysing, and improving performance in all aspects of an organization[2]

Address recurring and systemic issues to incorporate lessons learned into future activities



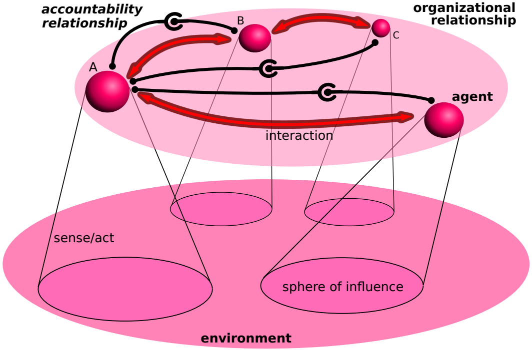
Accountability: a definition

"[A] social relationship in which an actor feels an obligation to explain and to justify his or her conduct to some significant other." **Bovens**

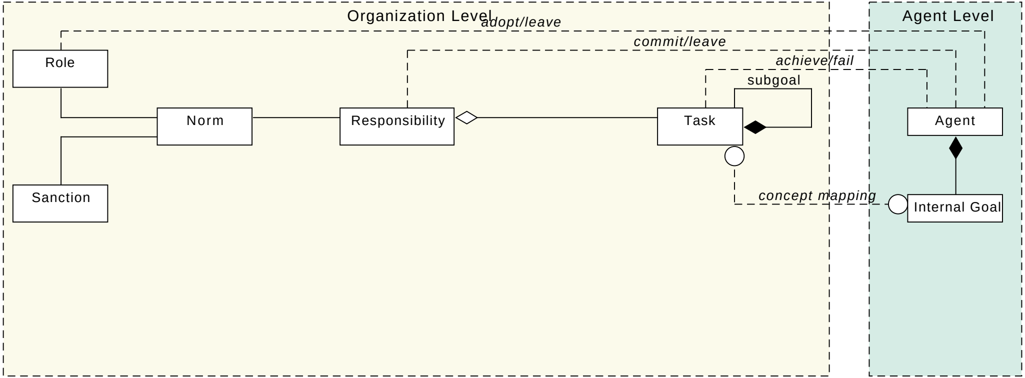
[2] "The UNDP accountability system, Accountability framework and oversight policy", United Nations, 2008, DP/2008/16/Rev.1.

In our work

- A formal description of robustness in multiagent organization (MAO): specify a proper treatment of possible perturbations
- A notion of accountability along two dimensions:
  - **normative dimension**
  - **structural dimension**
- An illustration in JaCaMo **JACAMO**



Norms, Roles, Responsibilities, Sanction, Tasks



Accountability as a means for robustness in MAS

- The current design methodologies for MAS fall short in addressing robustness in a systematic way at design time.

Accountability

We exploit the notion of **accountability Garfinkel1967**; **Grant05**; **Dubnick04**; **URANIA16**; **MOCA2019** as a mechanism for building feedback/reporting frameworks, similarly to what is often done in human organizations **UN**.

Accountability

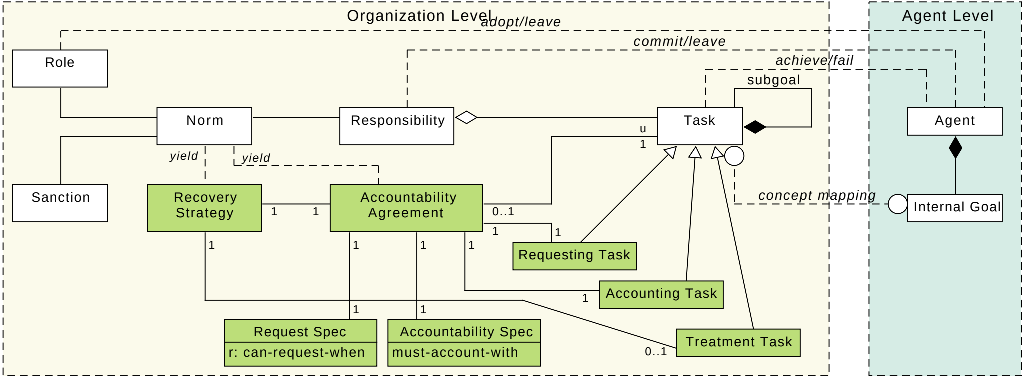
A relationship between two parties:

- One of the parties (the "**account taker**" or *a-taker*) can legitimately ask, under some agreed conditions, to the other party an **account** about a process of interest
- the other party (the "**account giver**" or *a-giver*) is legitimately required to provide the account to the a-taker

The two dimensions of accountability

1. **Normative dimension** → Legitimacy of asking and availability to provide accounts
2. **Structural dimension** → For being accountable about a process, an agent must have control over that process and have awareness of the situation it will account for

Recovery Strategies, Accountability Agreements, Request and Account Specs



JaCaMo Extensions

We present two extensions of the **JaCaMo**[3] agent platform to realize **robust MAS applications**

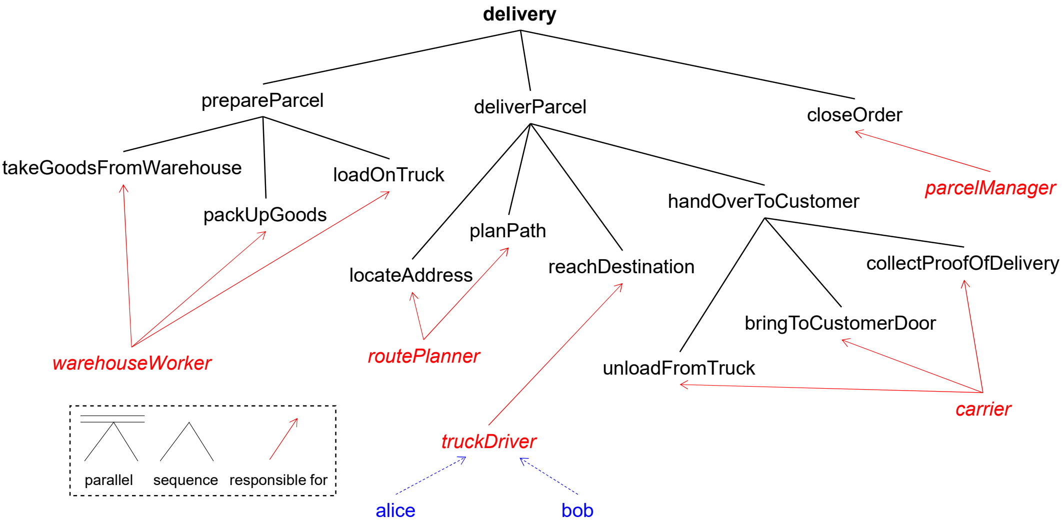
- **Accountability**
- **Exception Handling**

The proposed mechanisms rely on abstractions that are seamlessly integrated with organizational concepts, like:

- **Responsibilities**
- **Goals**
- **Norms**

[3] `http://jacamo.sourceforge.net/`

Example: parcel delivery



Example: parcel delivery (continued)

Consider the following scenario:

1. Alice and Bob have to meet at the same destination
2. Alice reaches the destination before Bob, and wants to investigate Bob's delay
3. Alice **requests Bob an account** about its delay
4. Bob answers with **reason for the delay**, e.g., a list of encountered **closed roads**
5. Alice can **leverage this information** to adjust the route in future deliveries

Specifying accountability

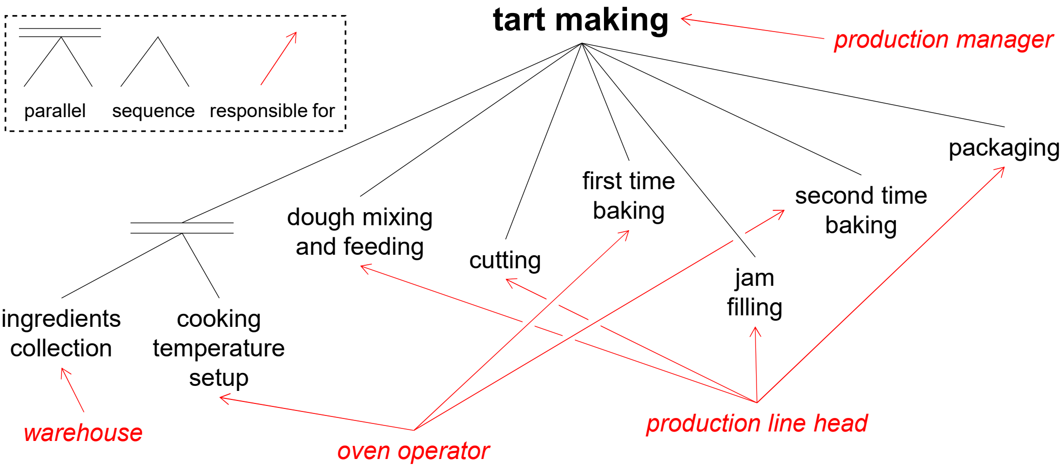
```
<notification-policy id="aaDelivery" target="reachDestination" condition="true" type="accountability">
    <report id="delay">
        <argument id="reason" arity="1" />
        <argument id="roads" arity="1" />
        <requesting-goal id="requestDelayReason" min="1" />
        <accounting-goal id="reportDelayReason" min="1" />
        <!-- <treatment-goal id="updateLocalMap"/> -->
        <!-- <treatment-goal id="updateGlobalMap"/> -->
    </report>
</notification-policy>

<mission id="mTruckDriver" min="2" max="2">
    <goal id="reachDestination" />
    <goal id="requestDelayReason" />
    <goal id="reportDelayReason" />
</mission>
```

Exception handling: anticipating and responding to failures

When a failure in achieving some goal occurs:

- Affected agents should be notified
- Alternative actions must be put in place
  - To do this, contextual information concerning the failure could be required
  - Such information may be visible only to some agents
  - The agent possessing this information may differ from the one possessing the capability to recover

Example: cookies



Example: cookies (continued)

Suppose goal `ingredientsCollection` fails due to the shortage of some ingredients
To overcome the issue (**reconfigure the organization**)

1. The warehouse should notify the production line about the shortage: e.g., what are the *missing ingredients*
2. The production line head should react by ordering a restock and by using some alternative ingredients

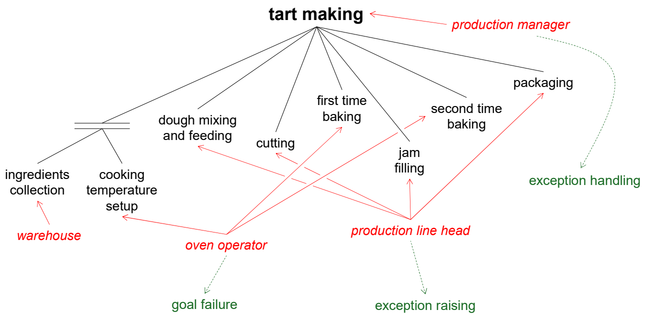
Specifying exception handling

```
<notification-policy id="np1" target="ingredientsCollection" condition="failed(_, ingredientsCollection )" type="exception">
    <report id="ingredientsShortage">
        <argument id="missingFillings" arity="1" />
        <raise-goal id="notifyIngredientsShortage"/>
        <handle-goal id="dealWithIngredientsShortage" />
    </report>
</notification-policy>

<mission id="mWarehouse" min="1" max="1">
    <goal id="ingredientsCollection" />
    <goal id="notifyIngredientsShortage" />
</mission>
<mission id="mProductionLineHead" min="1" max="1">
    <goal id="doughMixingAndFeeding" />
    <goal id="cutting" />
    <goal id="jamFilling" />
    <goal id="packaging" />
    <goal id="dealWithIngredientsShortage" />
</mission>
```

Exception raising and role separation

The agent who should raise an exception is not necessarily the one responsible for the failed goal

- Consider a failure in one of the goals assigned to the oven operator, due to an oven malfunctioning
- The failure could trigger an inspection by the production line head, which then raises the exception
- The exception is handled by the production manager



Benefits of the framework

- **High cohesion:** Tasks assigned to agents with the right capabilities
- **Low coupling:** Interaction through well-defined structured channels
- **Separation of concerns:** Business logic $\neq$ accountability and exception handling logic
- **Support for decentralization:** Each agent contributes from its local perspective

Where is the code?

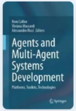
- Accountability and exception handling extensions will soon be released as official extensions to JaCaMo
- They can be currently downloaded at https://github.com/moise-lang/moise, under the exceptions branch
- the following paper provides examples an programming patterns

Home > Agents and Multi-Agent Systems Development > Chapter

Robust Multi-Agent Systems in JaCaMo: A Practical Guide to the Use of Exception Handling and Accountability

Chapter | First Online: 08 January 2026

pp 231–259 | Cite this chapter



Agents and Multi-Agent Systems Development

Matteo Baldoni, Cristina Baroglio, Roberto Micalizio & Stefano Tedeschi

Questions?

References

Baldoni, Matteo et al. **"Reimagining Robust Distributed Systems through Accountable MAS".** In: *IEEE Internet Computing* 25.6 (2021). DOI: 10.1109/MIC.2021.3115450. URL: https://doi.org/10.1109/MIC.2021.3115450.

— .**"Robustness Based on Accountability in Multiagent Organizations".** In: *Proceedings of the 20th International Conference on Autonomous Agents and MultiAgent Systems*. Ed. by Ulle Endriss et al. AAMAS '21. International Foundation for Autonomous Agents and Multiagent Systems, 2021, pp. 142–150. URL: http://www.ifaamas.org/Proceedings/aamas2021/pdfs/p142.pdf.

— .**"Exception Handling as a Social Concern".** In: *IEEE Internet Computing* 26.6 (2022), pp. 33–40. DOI: 10.1109/MIC.2022.3216272.

— .**"Accountability in multi-agent organizations: from conceptual design to agent programming".** In: *Autonomous Agents and Multi-Agent Systems* 37.1 (2023), pp. 1–37.

— .**"Robust Multi-Agent Systems in JaCaMo: A Practical Guide to the Use of Exception Handling and Accountability".** In: *Agents and Multi-Agent Systems Development: Platforms, Toolkits, Technologies*. Ed. by Rem Collier, Viviana Mascardi, and Alessandro Ricci. Springer Nature Switzerland, 2026, pp. 231–259. ISBN: 978-3-032-01082-7. DOI: 10.1007/978-3-032-01082-7_9.

"ISO/IEC/IEEE International Standard - Systems and software engineering – Vocabulary". In: *ISO/IEC/IEEE 24765:2010(E)* (2010), pp. 1–418.